ID NO: 2400030040

Name: N. Sri Aravind

#SKILL - 4 ->Spring Dependency Injection – Constructor & Setter Injection

Prerequisites:

- **Basic knowledge of Java**
- **General Idea on Spring Framework • Basic Idea on DI & IoC**

A training institute wants to automate the management of student information in a Spring application. The system should inject details such as studentId, name, course, and academic year into objects without manually assigning values. This Skill demonstrates Constructor Injection and Setter Injection using both XML and Annotation configurations.

Tasks:

1. **Create a Java class named Student with fields: studentId, name, course, year.**
2. **Create a Constructor that accepts all fields and assigns values.**
3. **Create Setter methods for at least two fields (example: course, year).**
4. **Configure the Student bean using: a. XML Configuration - The student should take: • a POJO class • an XML configuration file • a main class to load the container b. Annotation Configuration - The student should take: • a POJO class • a Java class with necessary annotations • a main class to load the container**
5. **Create a Spring configuration file (XML or Java-based).**
6. **Write a main class to load the Spring IoC container.**
7. **Retrieve the Student bean and print the injected values.**
8. **Push the Spring (core) project to GitHub**

Program Modules:

1)AnnotationMainApp.java

```
package com.klu.anno;

import org.springframework.context.ApplicationContext;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
import com.klu.config.AppConfig;
import com.klu.model.StudentAnno;

public class AnnotationMainApp {

    public static void main(String[] args) {

        ApplicationContext context =
                new AnnotationConfigApplicationContext(AppConfig.class);

        StudentAnno student = context.getBean(StudentAnno.class);
        System.out.println(student);
    }
}
```

2)AppConfig.java

```
package com.klu.config;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "com.klu")
public class AppConfig {
}
```

3)Student.java

```
package com.klu.model;

public class Student {

    private int studentId;
```

```
    private String name;
    private String course;
    private int year;

    // Constructor Injection
    public Student(int studentId, String name, String course, int year) {
        this.studentId = studentId;
        this.name = name;
        this.course = course;
        this.year = year;
    }

    // Setter Injection
    public void setCourse(String course) {
        this.course = course;
    }

    public void setYear(int year) {
        this.year = year;
    }

    @Override
    public String toString() {
        return "Student{" +
                "studentId=" + studentId +
                ", name='" + name + '\'' +
                ", course='" + course + '\'' +
                ", year=" + year +
                '}';
    }
}
```

4)StudentAnno.java

```
   package com.klu.model;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
@Component
public class StudentAnno {

   private int studentId;
   private String name;
   private String course;
   private int year;
   // Constructor Injection using @Value
   public StudentAnno(
         @Value("201") int studentId,
         @Value("Aravind") String name,
         @Value("Spring Boot") String course,
         @Value("2024") int year) {
      this.studentId = studentId;
      this.name = name;
      this.course = course;
      this.year = year;
   }
   // Setter Injection
   @Value("Advanced Spring")
   public void setCourse(String course) {
      this.course = course;
   }

   @Value("2025")
```

```
    public void setYear(int year) {
        this.year = year;
    }

    @Override
    public String toString() {
        return "StudentAnno {" +
                "studentId=" + studentId +
                ", name='" + name + '\'' +
                ", course='" + course + '\'' +
                ", year=" + year +
                '}';
    }
}
```

5)Bean.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="student" class="com.klu.model.Student">
        <!-- Constructor Injection -->
        <constructor-arg value="101"/>
        <constructor-arg value="Charan"/>
        <constructor-arg value="Spring Framework"/>
        <constructor-arg value="2025"/>
        <!-- Setter Injection -->
        <property name="course" value="Spring Core"/>
        <property name="year" value="2026"/>
    </bean>
</beans>
```

Output:

```
<terminated> skill4 - AnnotationMainApp [Spring Boot App] C:\Program Files\Java\jdk-24\bin\javaw.exe  (6 Feb 2026, 11:4
StudentAnno {studentId=201, name='Aravind', course='Advanced Spring', year=2025}
```

Git Repository:

https://github.com/Aravind2k6/2400030040-Fsad-WorkBook.git

VIVA QUESTIONS:

1. What is Dependency Injection (DI) in Spring?

A. Dependency Injection (DI) in Spring is a design pattern where the Spring framework automatically provides required objects (dependencies) to a class instead of the class creating them itself.

2. What is the difference between Constructor DI and Setter DI?

A. Constructor DI: Dependencies are injected through a class constructor (used for mandatory dependencies).

Setter DI: Dependencies are injected through setter methods (used for optional or changeable dependencies).

3. What is the purpose of the IoC container?

A. The IoC (Inversion of Control) container in Spring is responsible for creating, configuring, and managing the lifecycle of application objects (beans) and injecting their dependencies automatically.

4. Why is DI preferred over manual object creation?

A . Dependency Injection (DI) is preferred over manual object creation because it reduces tight coupling, improves code maintainability and testability, and allows better management of object lifecycles.

5. Can Constructor Injection and Setter Injection be used together?

A. Yes, Constructor Injection and Setter Injection can be used together in the same class — typically constructor injection is used for mandatory dependencies, and setter injection is used for optional dependencies.